

AI Assisted Coding Assignment- 11.2

2303A51886 : 30 SAI SPURTHI. B 10.03.2026

Lab 11 – Data Structures with AI: Implementing Fundamental Structures

Lab Objectives

- Use AI to assist in designing and implementing fundamental data structures in Python.
- Learn how to prompt AI for structure creation, optimization, and documentation.
- Improve understanding of Lists, Stacks, Queues, Linked Lists, Trees, Graphs, and Hash Tables.
- Enhance code quality with AI-generated comments and performance suggestions.

Task Description -1 – (Stack Using AI Guidance)

- Task: With the help of AI, design and implement a Stack data structure supporting basic stack operations.

Expected Output:

- A Python Stack class supporting push, pop, peek, and empty-check operations with proper documentation.


```
Lab-ASS-11.2.py > ...
38 #Task2
39 #Write a Python program to implement a Queue data structure
40 #following FIFO (First In First Out).
41 #Include methods enqueue(), dequeue(), front(), and size().
42 #Add comments and example usage.
43 class Queue:
44     def __init__(self):
45         self.queue = []
46     def enqueue(self, item):
47         # add element to rear
48         self.queue.append(item)
49     def dequeue(self):
50         # remove element from front
51         if not self.is_empty():
52             return self.queue.pop(0)
53         return "Queue is empty"
54     def front(self):
55         # get front element
56         if not self.is_empty():
57             return self.queue[0]
58         return "Queue is empty"
59     def size(self):
60         return len(self.queue)
61     def is_empty(self):
62         return len(self.queue) == 0
63 # Example usage
64 q = Queue()
65 q.enqueue(5)
66 q.enqueue(15)
67 q.enqueue(25)
68 print("Front element:", q.front())
69 print("Removed:", q.dequeue())
70 print("Queue size:", q.size())
71
72 Queue is empty: False
73 Front element: 5
74 Removed: 5
75 Queue size: 2
76 Removed element: 30
```

Task Description -3 -(Singly Linked List Construction)

- Task: Utilize AI to build a singly linked list supporting insertion and traversal.

Expected Output:

- Correctly functioning linked list with node creation, insertion logic, and display functionality.

```
Lab-13-ASS.py  Lab-ASS-13.5.py  Lab-ASS-11.2.py X  data1.txt  data2.txt  library.py  library
Lab-ASS-11.2.py > ...
74 #Task3
75 #Write a Python program to implement a Singly Linked List.
76 #Each node should contain data and a reference to the next node.
77 #Include methods for inserting a node and displaying the list.
78 #Add comments and example usage.
79 class Node:
80     def __init__(self, data):
81         # store data and next pointer
82         self.data = data
83         self.next = None
84 class LinkedList:
85     def __init__(self):
86         # initialize head of list
87         self.head = None
88     def insert(self, data):
89         # create new node
90         new_node = Node(data)
91         # if list is empty, make new node head
92         if self.head is None:
93             self.head = new_node
94             return
95         # traverse to the end of list
96         temp = self.head
97         while temp.next:
98             temp = temp.next
99         # insert node at the end
100        temp.next = new_node
101    def display(self):
102        # print linked list elements
103        temp = self.head
104        while temp:
105            print(temp.data, end=" -> ")
106            temp = temp.next
107        print("None")
108 # Example usage
109 ll = LinkedList()
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
Python + - [ ] [ ] [ ]
PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING> & "C:\Program Files\Python38\python.exe" "c:/Users/spurthi bellamkonda/OneDrive/Desktop/AI CODING/Lab-ASS-11.2.py"
Removed: 5
Queue size: 2
Removed: 5
Queue size: 2
Queue size: 2
Linked List:
10 -> 20 -> 30 -> None
In-order Traversal:
20 30 40 50 70 Search 20: True
Key deleted
```

Task Description -4 – (Binary Search Tree Operations):

- Task: Implement a Binary Search Tree with AI support focusing on insertion and traversal.

Expected Output:

- BST program with correct node insertion and in-order traversal output.

```
Lab-13-ASS.py Lab-ASS-13.5.py Lab-ASS-11.2.py X data1.txt data2.txt library.py library
Lab-ASS-11.2.py > ...
118 #Task4
119 #Write a Python program to implement a Binary Search Tree (BST).
120 #The BST should support insertion and in-order traversal.
121 #Add comments and example usage.
122 class Node:
123     def __init__(self, value):
124         # initialize node with value and left/right children
125         self.value = value
126         self.left = None
127         self.right = None
128 class BST:
129     def insert(self, root, value):
130         # insert a new node into the BST
131         if root is None:
132             return Node(value)
133         if value < root.value:
134             root.left = self.insert(root.left, value)
135         else:
136             root.right = self.insert(root.right, value)
137         return root
138     def inorder(self, root):
139         # perform in-order traversal
140         if root:
141             self.inorder(root.left)
142             print(root.value, end=" ")
143             self.inorder(root.right)
144 # Example usage
145 tree = BST()
146 root = None
147 root = tree.insert(root, 50)
148 root = tree.insert(root, 30)
149 root = tree.insert(root, 70)
150 root = tree.insert(root, 40)
151 root = tree.insert(root, 20)
152 # In-order Traversal
153 tree.inorder(root)
154 # Search
155 search_value = 20
156 if tree.search(root, search_value):
157     print("Search", search_value, ":", True)
158 else:
159     print("Search", search_value, ":", False)
160 # Delete
161 delete_value = 10
162 root = tree.delete(root, delete_value)
163 # In-order Traversal after deletion
164 tree.inorder(root)
165 # Search after deletion
166 search_value = 20
167 if tree.search(root, search_value):
168     print("Search", search_value, ":", True)
169 else:
170     print("Search", search_value, ":", False)
```

```
In-order Traversal:
20 30 40 50 70 Search 20: True
Key deleted
10 -> 20 -> 30 -> None
In-order Traversal:
20 30 40 50 70 Search 20: True
```

Task Description -5 – (Hash Table Implementation)

- Task: Create a hash table using AI with collision handling

Expected Output:

- Hash table supporting insert, search, and delete using chaining or open

```
Lab-ASS-11.2.py > ...
157 #Task5
158 #Write a Python program to implement a Hash Table.
159 #Use chaining to handle collisions.
160 #Include methods insert(), search(), and delete().
161 #Add comments and example usage.
162 class HashTable:
163     def __init__(self, size):
164         # create hash table with empty lists
165         self.size = size
166         self.table = [[] for _ in range(size)]
167     def hash_function(self, key):
168         # simple hash function
169         return key % self.size
170     def insert(self, key):
171         # insert key into hash table
172         index = self.hash_function(key)
173         self.table[index].append(key)
174     def search(self, key):
175         # search for key in hash table
176         index = self.hash_function(key)
177         if key in self.table[index]:
178             return True
179         return False
180     def delete(self, key):
181         # delete key from hash table
182         index = self.hash_function(key)
183         if key in self.table[index]:
184             self.table[index].remove(key)
185             return "Key deleted"
186         return "Key not found"
187 # Example usage
188 h = HashTable(10)
189 h.insert(10)
190 h.insert(20)
191 h.insert(15)
192 print("Search 20:", h.search(20))
```

Ln 161, Col 33 Spaces: 4 UTF-8 CRLF Python Python 3.8 (64-bit) Go Live

```
Key deleted
10 -> 20 -> 30 -> None
In-order Traversal:
20 30 40 50 70 Search 20: True
Key deleted
In-order Traversal:
20 30 40 50 70 Search 20: True
Key deleted
20 30 40 50 70 Search 20: True
Key deleted
Key deleted
Search 20 after deletion: False
PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING>
```